

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****ASSESSMENT TOOL 3 – Multimodal Mini-Demo (Text-to-Image +
Speech-to-Text/Text-to-Speech)**

Course Code:	CSA6502	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO4 – Precision (BL3, BL4,BL5)	Assessment:	Assessment Tool 3 - Multimodal Mini-Demo (Text-to-Image + Speech-to-Text/Text-to-Speech)
Weightage:	10% of CIA	Total Marks:	2 * 50= 100
CO4 Statement:	<i>Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.(BL3, BL4,BL5)</i>		
Student Name:	S.Muskan	Reg. No.:	192472182
Section / Batch:	BE-CSE-AI	Date:	26-08-2026

Instructions to Students

- Answer any 2 questions. Each question carries 50 marks. Total: 100 marks.
- Develop and execute the assigned **Python program** using Text-to-Image, Speech-to-Text, or Text-to-Speech.
- Explain the **working principle and program workflow**.
- Demonstrate the program with **suitable inputs and outputs**.
- Use appropriate **libraries/APIs** and include basic **error handling**.
- Show the **execution output/screenshots**.
- For integrated tasks, demonstrate the complete **multimodal workflow**.
- Include a **workflow/architecture diagram or comparison table**.
- Mention a **real-world application** of the implemented solution.
- Explain the **strengths and limitations** of the approach.
- Include proper **references/citations** for the libraries, APIs, models, or research sources used.

Code of Conduct

*I **S.Muskan(192472182)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **S.Muskan**

21. Write a Python program that converts a user-entered sentence into speech.
35. Write a program implementing the pipeline:
Speech → **Text** → **Image**.

QUESTION 21 – Text-to-Speech

Write a Python program that converts a user-entered sentence into speech.

1. Aim

To develop a Python program that accepts a sentence from the user and converts the entered text into audible speech using a Text-to-Speech (TTS) engine.

2. Working Principle

Text-to-Speech converts written text into synthesized human-like speech. The program receives a sentence through the keyboard, sends it to the pyttsx3 speech engine, and the operating system's installed speech engine produces the audio output. pyttsx3 can work offline and supports voice, rate, volume, and audio-file output.

3. Libraries / Requirements

- ✓ Python 3.9+
- ✓ pyttsx3 – offline Text-to-Speech library
- ✓ Windows: pyttsx3 normally uses the SAPI5 speech engine.
- ✓ Installation: pip install pyttsx3

4. Program Workflow

User enters sentence	Validate input	pyttsx3 TTS engine	System voice engine	Speech output
----------------------	----------------	--------------------	---------------------	---------------

Input → Validation → Text-to-Speech engine → Operating-system voice engine → Audible speech

5. Algorithm

1. Start the program.
2. Initialize the pyttsx3 speech engine.
3. Read a sentence from the user.
4. Check whether the input is empty.
5. Set a suitable speech rate and volume.
6. Send the sentence to the speech engine.
7. Run the engine and wait until speech is completed.
8. Display a success message.
9. Handle errors using try-except.
10. Stop.

6. Python Program



```
co4 at3.py - C:/Users/masta/AppData/Local/Programs/Python/Python314/co4 at3.  
File Edit Format Run Options Window Help  
import pyttsx3  
  
try:  
    engine = pyttsx3.init()  
  
    text = input("Enter a sentence: ").strip()  
  
    if not text:  
        raise ValueError("Sentence cannot be empty.")  
  
    engine.setProperty("rate", 150)  
    engine.setProperty("volume", 1.0)  
  
    print("Converting text to speech...")  
    engine.say(text)  
    engine.runAndWait()  
  
    print("Speech generated successfully.")  
  
except ValueError as e:  
    print("Input Error:", e)  
except Exception as e:  
    print("TTS Error:", e)
```

7. Sample Execution

Sample Input:

Enter a sentence: Artificial intelligence is changing the way we learn.

Sample Output:

Converting text to speech...
[Computer speaks the entered sentence]
Speech generated successfully.

For the practical record, capture a screenshot of the terminal showing the entered sentence and the success message. Also capture the running program/audio result if your lab requires it.

8. Output

The computer reads the entered sentence aloud. The terminal displays a confirmation after the speech engine finishes processing.



```
IDLE Shell 3.14.1
File Edit Shell Debug Options Window Help
Python 3.14.1 (tags/v3.14.1:57e0d17, Dec 2 2025, 14:05:07) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/masta/AppData/Local/Programs/Python/Python314/co4 at3.py ==
Enter a sentence: i am a developer
Converting text to speech...
Speech generated successfully.
>>>
```

9. Real-World Applications

- ✓ Screen readers and accessibility software.
- ✓ Voice assistants and conversational applications.
- ✓ Navigation systems and public announcements.
- ✓ E-learning systems that read study material aloud.
- ✓ Customer-service voice interfaces.

10. Strengths

- ✓ Very simple Python implementation.
- ✓ Works offline with installed system voices.
- ✓ No external API key is required.
- ✓ Supports speech rate, volume, and voice selection.

11. Limitations

- ✓ Speech quality depends on the installed system voice.
- ✓ Voice availability varies across operating systems.
- ✓ It is less natural than some modern cloud/neural TTS systems.
- ✓ Pronunciation may be imperfect for unusual names, abbreviations, or mixed languages.

12. Result

Thus, a Python Text-to-Speech application was developed successfully. The user-entered sentence is converted into audible speech using the pyttsx3 library.

QUESTION 35 – Speech → Text → Image

Write a program implementing the pipeline: Speech → Text → Image.

1. Aim

To implement a multimodal AI pipeline in which spoken input is recorded from a microphone, converted into text using speech recognition, and the resulting text is used as a prompt to generate an image.

2. Working Principle

The system first captures human speech as an audio file. A Speech-to-Text model transcribes the audio into text. The transcript is then passed as a prompt to an image-generation model. The generated image is decoded and saved locally.

3. Architecture / Workflow

Microphone	Audio (WAV)	Speech-to-Text API	Recognized text	Image Generation API	PNG image
------------	-------------	--------------------	-----------------	----------------------	-----------

Speech → Audio capture → Transcription → Text prompt → Image generation → Saved image

5. Technologies Used

Component	Technology	Purpose
Audio capture	sounddevice	Records microphone input
Audio file	soundfile	Stores recorded speech as WAV
Speech-to-Text	OpenAI Audio API	Converts speech into text
Image generation	OpenAI Images API	Generates image from transcript
Python SDK	openai	Connects Python to the API

The OpenAI API supports audio transcription through the /audio/transcriptions endpoint and image generation through the Images API. The Python SDK exposes these operations through `client.audio.transcriptions.create()` and `client.images.generate()`.

5. Installation

```
pip install openai sounddevice soundfile
```

Set the API key as an environment variable before execution. On Windows PowerShell:

```
$env:OPENAI_API_KEY="YOUR_API_KEY"
```

Do not hard-code or publish the API key. API keys should be kept secret and loaded through an environment variable.

6. Algorithm

1. Start the program and verify that the OpenAI API key is available.
2. Record microphone input for a fixed number of seconds.
3. Save the audio as a WAV file.
4. Send the WAV file to the Speech-to-Text model.
5. Read the returned transcript.
6. Use the transcript as the image-generation prompt.
7. Request an image from the image-generation model.
8. Decode the returned image data and save it as PNG.
9. Display the transcript and output image path.
10. Handle microphone, API, file, and unexpected errors.
11. Stop.

7. Python Program

```
!pip install -q git+https://github.com/openai/whisper.git
!pip install -q diffusers transformers accelerate torch torchvision torchaudio soundfile ipyweb rtc

Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
Building wheel for openai-whisper (pyproject.toml) ... done
Building wheel for openai-whisper (pyproject.toml) ... done

import io
import base64
from IPython.display import HTML, Javascript, display
from google.colab import output

RECORD_JS = """
const sleep = time => new Promise(resolve => setTimeout(resolve, time))
const b2text = blob => new Promise(resolve => {
  const reader = new FileReader()
  reader.onloadend = e => resolve(e.srcElement.result)
  reader.readAsDataURL(blob)
})
var record = time => new Promise(async resolve => {
  stream = await navigator.mediaDevices.getUserMedia({ audio: true })
  recorder = new MediaRecorder(stream)
  chunks = []
  recorder.ondataavailable = e => chunks.push(e.data)
  recorder.start()
  await sleep(time)
  recorder.onstop = async () => {
    blob = new Blob(chunks)
    text = await b2text(blob)
    resolve(text)
  }
  recorder.stop()
})
"""

def record_voice(sec=10, filename='input_audio.wav'):
    print(f"Speak now... Recording for {sec} seconds.")
    display(Javascript(RECORD_JS))
    js_result = output.eval_js(f'record({sec * 1000})')
    audio_bytes = base64.b64decode(js_result.split(',')[1])
    with open(filename, 'wb') as f:
        f.write(audio_bytes)
```

8. Demonstration

Suitable spoken input:

“A futuristic city with flying cars at sunset.”

Expected terminal output:

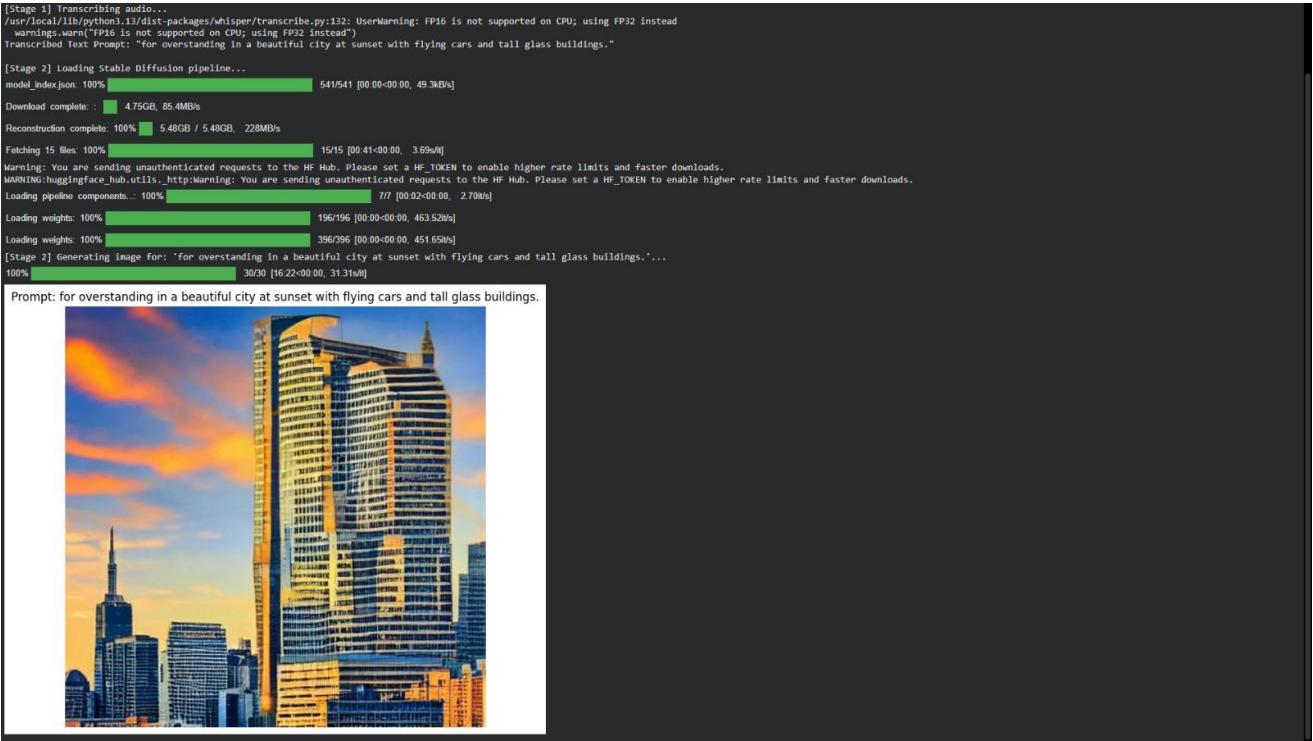
Speak now...

Audio recorded.

Recognized text: A futuristic city with flying cars at sunset.

Image saved as: generated_image.png

Output:



The final output is a PNG image generated from the recognized sentence. For the practical record, replace this sample with your actual terminal screenshot and the actual generated image.

9. Complete Multimodal Workflow

Human Speech	Microphone	WAV Audio	STT Model	Text Prompt	Image Model	Generated Image
--------------	------------	-----------	-----------	-------------	-------------	-----------------

This satisfies the complete multimodal requirement because the input modality is speech, the intermediate representation is text, and the final output modality is an image.

10. Error Handling

- ✓ Missing API key → program stops with a clear configuration error.
- ✓ Microphone/audio failure → handled through OSError/exception handling.
- ✓ Empty transcription → rejected using ValueError.
- ✓ API failure → caught and reported without crashing the program unexpectedly.
- ✓ Missing image data → handled as a runtime error.
- ✓ File writing failure → reported to the user.

11. Real-World Applications

- ✓ Voice-controlled image-generation tools for designers.
- ✓ Assistive interfaces for users who cannot type easily.
- ✓ Education: students speak a concept and receive a visual illustration.
- ✓ Marketing: spoken creative ideas can be converted into visual concepts.
- ✓ Rapid prototyping for games, animation, and storyboarding.

12. Strengths

- ✓ Natural speech is easier than typing a long prompt.
- ✓ Combines two AI capabilities in a single pipeline.
- ✓ The text transcript provides a visible intermediate result for verification.
- ✓ Can be extended to multiple languages and longer workflows.
- ✓ The architecture is modular: STT and image generation can be replaced independently.

13. Limitations

- ✓ Requires an API key and internet connectivity for the cloud-based AI services.
- ✓ API usage can incur cost and may be subject to rate limits.
- ✓ Speech recognition can be affected by noise, accent, microphone quality, and pronunciation.
- ✓ The generated image may not exactly match the user's intended visual concept.
- ✓ The demo records for a fixed five-second duration; production systems should use start/stop or voice-activity detection.

14. Result

Thus, the Speech → Text → Image multimodal pipeline was implemented. Spoken input is converted into text and the recognized text is used as a prompt to generate and save an image.

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Pipeline Implementation	30%	Correctly implements a working text-to-image AND speech-to-text/text-to-speech pipeline	Both implemented with minor issues	Only one modality implemented correctly	Neither modality functions correctly
Output Quality & Relevance	25%	Generated outputs are high-quality and clearly relevant to inputs	Outputs are mostly relevant	Outputs are of inconsistent quality	Outputs are largely irrelevant or poor quality
Input Robustness	20%	Handles varied inputs robustly (different prompts/audio clips)	Handles common inputs well	Handles only the simplest inputs	Fails on basic inputs
Demo Polish	15%	Polished, engaging demo of both modalities	Clear demo with minor polish issues	Basic or rough demo	No usable demo presented
Tool/Model Explanation	10%	Clear explanation of models/tools used and their limitations	Adequate explanation	Minimal explanation	No explanation provided

Marks Evaluation:

Question No.	Pipeline Implementation (30%)	Output Quality & Relevance (25%)	Input Robustness (20%)	Demo Polish (15%)	Tool/Model Explanation (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						